

ЛЕКЦІЯ 3

РОЗРОБЛЕННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ ДЛЯ СИСТЕМИ КЕРУВАННЯ БПЛА, ЩО ВИКОНУЄ ФУНКЦІЇ ПОШУКУ ТА ІДЕНТИФІКАЦІЇ ОБ'ЄКТІВ НА ОСНОВІ МЕТОДІВ ШТУЧНОГО ІНТЕЛЕКТУ

1. ПРИНЦИПИ ПОШУКУ ТА ІДЕНТИФІКАЦІЇ ОБ'ЄКТІВ

2. ТЕХНІЧНІ АСПЕКТИ ПРОЄКТУ

3. ПРОГРАМНА РЕАЛІЗАЦІЯ ПРОЄКТУ

4. МУЛЬТИМОДАЛЬНЕ РОЗПІЗНАВАННЯ

1. ПРИНЦИПИ ПОШУКУ ТА ІДЕНТИФІКАЦІЇ ОБ'ЄКТІВ

Принципи пошуку та ідентифікації об'єктів на основі методів штучного інтелекту (ШІ) базуються на поєднанні різних підходів до обробки та аналізу даних, таких як машинне навчання, комп'ютерний зір і нейронні мережі. Основні етапи цього процесу включають:

1. Збір та підготовка даних

- **Збір даних:** Для навчання моделей ШІ необхідно мати велику кількість даних, що містять об'єкти, які потрібно розпізнавати. Це можуть бути зображення, відео або інші форми даних, що містять характеристики об'єктів.
- **Анотація даних:** Важливо мати марковані дані, де об'єкти чітко позначені (наприклад, позначення на зображеннях). Це допомагає моделі навчитися розпізнавати об'єкти за їх ознаками.

- **Попередня обробка даних:** Можуть бути використані такі техніки, як нормалізація, масштабування, видалення шуму для покращення якості даних перед навчанням моделі.

2. Вибір моделі та навчання

- **Моделі машинного навчання:** Найпопулярніші методи включають нейронні мережі (особливо глибокі нейронні мережі, такі як згорткові нейронні мережі (CNN), що спеціалізуються на обробці зображень), методи класифікації, як-от дерева рішень, SVM тощо.
- **Навчання моделі:** Після вибору архітектури, модель навчається на даних, мінімізуючи помилки класифікації чи розпізнавання. В цьому процесі використовується функція втрат і методи оптимізації, такі як градієнтний спуск.

3. Екстракція ознак

- **Екстракція ознак:** Моделі ШІ, особливо глибокі нейронні мережі, здатні автоматично витягати релевантні ознаки з даних (наприклад, контури, текстури, кольори), які дозволяють їм краще ідентифікувати об'єкти.
- **Ручна екстракція ознак:** У традиційних методах (без глибокого навчання) можуть використовуватися алгоритми типу SIFT, HOG, які витягують ключові ознаки для подальшої обробки.

4. Класифікація або ідентифікація об'єкта

- **Класифікація:** Модель використовує витягнуті ознаки для визначення класу об'єкта (наприклад, розпізнавання об'єкта як людина, тварина або автомобіль).
- **Ідентифікація:** Крім загальної класифікації, можлива детальна ідентифікація конкретного об'єкта або суб'єкта на основі унікальних ознак (наприклад, обличчя людини).

5. Оцінка якості моделі

- **Точність:** Визначається відсотком правильно класифікованих об'єктів або ідентифікованих елементів.
- **Повнота та точність:** Додаткові метрики, такі як precision (точність) і recall (повнота), допомагають оцінити, наскільки добре модель справляється з класифікацією різних об'єктів.
- **Змішані матриці та криві ROC:** Використовуються для візуалізації та аналізу продуктивності моделі.

6. Реальне застосування

- **Комп'ютерний зір:** Розпізнавання об'єктів в зображеннях або відео, включаючи системи відеоспостереження, автономні транспортні засоби тощо.
- **Біометричні системи:** Розпізнавання облич, відбитків пальців та інших унікальних ознак людини.
- **Роботи і автономні системи:** Ідентифікація об'єктів у просторі для взаємодії з середовищем.

ШІ використовує моделі, які навчаються на основі *великої кількості даних* для автоматизації процесу пошуку та ідентифікації об'єктів з високою точністю та швидкістю.

2. ТЕХНІЧНІ АСПЕКТИ ПРОЄКТУ

1. Коротка мета проєкту

Розробити програмне забезпечення для системи керування БПЛА, що в реальному часі виконує пошук, локалізацію та ідентифікацію наземних/морських/повітряних об'єктів із застосуванням методів комп'ютерного зору та машинного навчання. Система повинна працювати як у автономному режимі (на борту), так і в супроводі з наземною станцією (телеметрія, карта, інтерфейс оператора).

2. Обсяг (scope)

- Сканування та виявлення об'єктів на відеопотоці з камер (*RGB, тепловізор* — опційно).
- Класифікація (ідентифікація типів об'єктів (людина, транспорт, споруда, тощо)).
- Локалізація об'єктів у геокоординатах (*прив'язка до карти*).
- Траєкторне слідування та відстеження цілей (*tracking*).
- Інтерфейс для оператора (*відео, маркери, повідомлення, можливість ручного керування*).

3. Основні функціональні модулі

1. Контролер польоту (*Flight controller interface*)

- Підключення до автопілота (*MAVLink*).
- Виконання команд (*takeoff, waypoint, loiter, RTL*).

2. Пайплайн зображення (*Vision pipeline*)

- Збір кадрів з камер, попередня обробка .
- Детекція об'єктів (*виявлення / сегментація*).
- Трекінг (ID-ніжка для кожного об'єкта).
- Геоприв'язка (перетворення зображення → GPS).

3. Планування місії / тактика пошуку

- Генерація шляхів
- Пріоритети зон та динамічні зміни місії при виявленні.

4. Компонент відстеження та прийняття рішень

- Автоматичні дії: наблизитись / сфотографувати / передати координати / відправити попередження.

5. Безпека / логування / менеджмент даних

- Шифрування каналів, доступи, анонімізація, політика зберігання відео.

4. Рекомендований технічний стек (*software & libraries*)

- **Комунікація з автопілотом** (*ArduPilot*)
- **Моделі детекції:** сучасні варіанти (*YOLO-family*);

- **Tracking** (*DeepSORT*)
- **Georeferencing**: використати матрицю камери + дані GPS для проєкції pixel→GPS.

5. Дані та навчання

- **Джерела даних**: публічні набори — *COCO* (загальні об'єкти),
- **Аугментації**: рандомні повороти, освітлення, шум, масштаб, рандомні кадрування, зміни спектра
- **Методика навчання**: початкове навчання на великих загальних наборах
- **Верифікація**: розділити дані на train/val/test по місіях

6. Метрики якості

- **Localization**: середня помилка позиціонування між справжніми GPS і обчисленими.
- **Надійність**: відсоток втрат зв'язку, пропуск кадрів.

7. Апаратні рекомендації (категорії)

- **Камери**: RGB з високою роздільністю; якщо потрібна нічна робота — тепловізор.
- **Автопілот**: сумісний *MAVLink*

8. Тестування й валідація

- **Симуляція**: *AirSim* для тестів без реального польоту.
- **Польотні випробування**: в контрольованих умовах, з поетапним розширенням зони/часу.
- **Regression testing** для моделей після донавчання.
- **Safety tests**: вихід із зони, втрати зв'язку, відновлення місії, аварійні сценарії.

3. ПРОГРАМНА РЕАЛІЗАЦІЯ ПРОЄКТУ

[CMM LEC 3-1]

```
import cv2
```

```
# Завантажуємо попередньо навчену нейромережу MobileNetSSD (детектор об'єктів)
net = cv2.dnn.readNetFromCaffe(
    "deploy.prototxt",
    "mobilenet_iter_73000.caffemodel"
```

```

)

# Класи, які може розпізнати модель
CLASSES = ["background", "aeroplane", "bicycle", "bird", "boat",
            "bottle", "bus", "car", "cat", "chair", "cow", "diningtable",
            "dog", "horse", "motorbike", "person", "pottedplant",
            "sheep", "sofa", "train", "tvmonitor"]

# Запуск вебкамери
cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()
    if not ret:
        break

    # Підготовка кадру для неймережі
    (h, w) = frame.shape[:2]
    blob = cv2.dnn.blobFromImage(cv2.resize(frame, (300, 300)), 0.007843, (300,
300), 127.5)

    net.setInput(blob)
    detections = net.forward()

    # Проходимо по знайденим об'єктам
    for i in range(detections.shape[2]):
        confidence = detections[0, 0, i, 2]

        if confidence > 0.5: # поріг впевненості
            idx = int(detections[0, 0, i, 1])
            box = detections[0, 0, i, 3:7] * [w, h, w, h]
            (startX, startY, endX, endY) = box.astype("int")

            label = f"{CLASSES[idx]}: {confidence*100:.1f}%"
            cv2.rectangle(frame, (startX, startY), (endX, endY), (0, 255, 0), 2)
            cv2.putText(frame, label, (startX, startY - 10),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)

    cv2.imshow("Object Identification", frame)

    if cv2.waitKey(1) & 0xFF == ord("q"): # вихід по клавіші q
        break

cap.release()
cv2.destroyAllWindows()

```

- Завантажені два файли та збережені у тій самій директорії, де знаходиться ваш Python-скрипт.

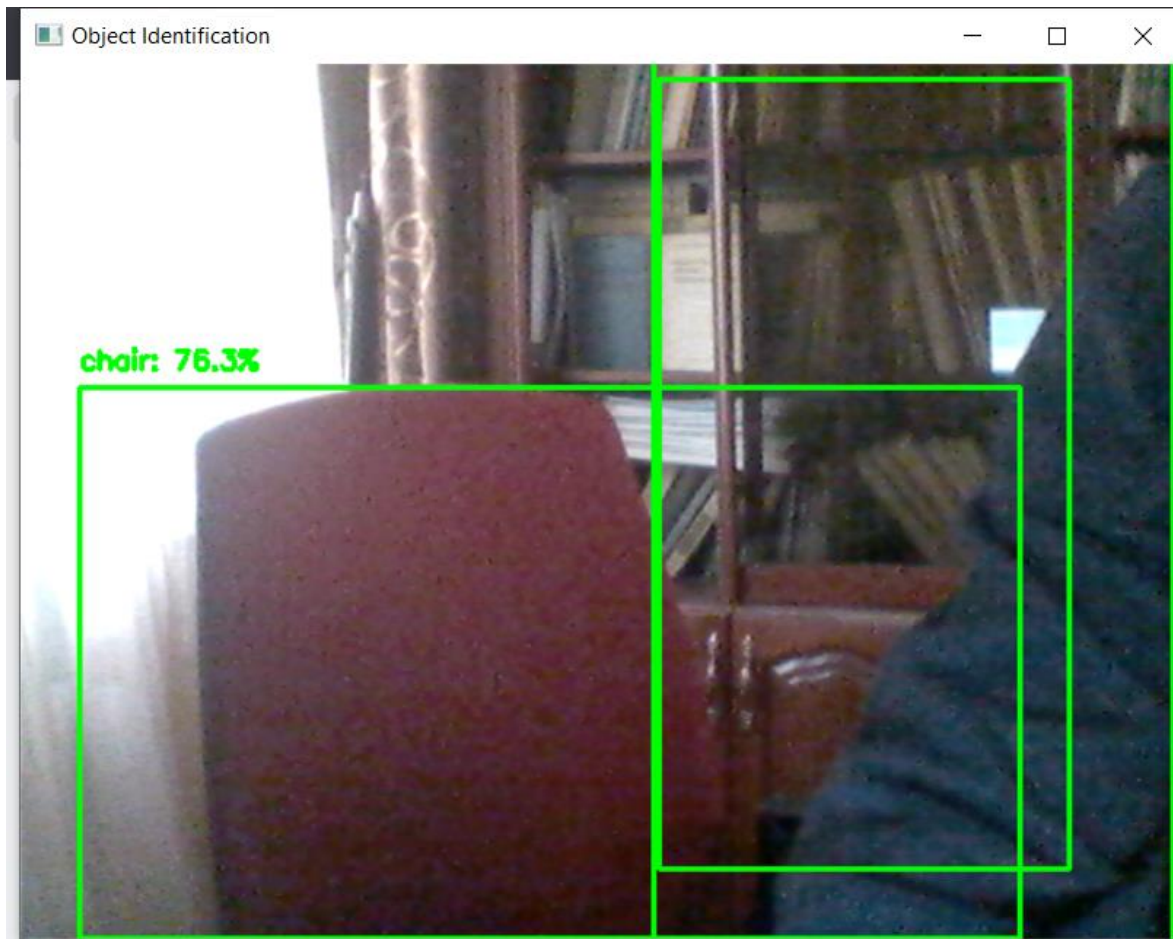
- У коді використовуємо ці файли для завантаження моделі:

```

# Завантаження попередньо натренованої моделі
MobileNet-SSD
net = cv2.dnn.readNetFromCaffe('deploy.prototxt',
'mobilenet_iter_73000.caffemodel')

```

- Тепер можемо використовувати модель для виявлення об'єктів у відео або зображеннях, як показано в попередньому коді.



4. МУЛЬТИМОДАЛЬНЕ РОЗПІЗНАВАННЯ

Системи штучного інтелекту для розпізнавання та класифікації об'єктів в реальному часі використовуються в різних сферах, від охорони безпеки до промисловості та автономного транспорту. Вони можуть працювати з даними різної якості, що означає, що їхні алгоритми повинні бути стійкими до шуму, розмитих зображень, змін освітлення, впливу засобів РЕБ та інших викликів.

Такі системи складаються з декількох ключових компонентів:

1. **Попередня обробка даних** – це етап, де відбувається зниження шуму, корекція контрастності та підвищення якості зображення, що допомагає покращити точність розпізнавання.

2. **Нейронні мережі** – для задач розпізнавання часто використовуються глибокі нейронні мережі, такі як згорткові нейронні мережі (CNN) для обробки зображень. CNN добре працюють з розпізнаванням різних об'єктів, як-от обличчя, транспортні засоби, товари на виробничій лінії тощо.

<https://cutt.ly/YrpT37fV>

Використання технологій мультимодального розпізнавання – щоб врахувати дані різної якості, використовуються моделі, які об'єднують дані з кількох джерел (наприклад, з камер, радарів, сенсорів глибини) і адаптують підхід до якості кожного каналу даних.

Системи класифікації з підтримкою самонавчання – ці моделі можуть покращуватися під час використання, збільшуючи точність розпізнавання завдяки зворотному зв'язку від користувачів.

Залежно від потреб, такі системи можуть працювати як локально на пристрої, так і відправляти дані на сервер для глибшої обробки.

Приклад програми для розпізнавання та класифікації об'єктів у реальному часі з використанням моделі YOLOv5, яка є більш сучасною, точною і швидшою порівняно з YOLOv3. Вона підтримує інтерфейс PyTorch та працює через бібліотеку torch.

[MS LEC 8-1]

```
import torch
import cv2

# Завантажуємо попередньо натреновану модель YOLOv5
model = torch.hub.load('ultralytics/yolov5', 'yolov5s',
pretrained=True)

# Підключення до камери
cap = cv2.VideoCapture(0)
if not cap.isOpened():
    print("Не вдалося підключитися до камери")
    exit()

while True:
    # Зчитування кадру з камери
    ret, frame = cap.read()
    if not ret:
        print("Не вдалося зчитати кадр з камери")
        break

    # Використання моделі для виявлення об'єктів на кадрі
    results = model(frame)

    # Отримуємо координати, клас і достовірність для
    кожного об'єкта
    for detection in results.xyxy[0]:
        x1, y1, x2, y2, confidence, class_id = map(int,
detection[:6])
        label = model.names[class_id] # Назва класу
```

```
confidence_text = f"{confidence * 100:.2f}%"

# Малюємо рамку навколо об'єкта
cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 255,
0), 2)

cv2.putText(frame, f"{label}: {confidence_text}",
(x1, y1 - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255,
0), 2)

# Відображаємо кадр з розпізнаними об'єктами
cv2.imshow("YOLOv5 Object Detection", frame)

# Затримка, щоб уникнути зависання
if cv2.waitKey(1) & 0xFF == ord('q'):
    break

# Звільнення ресурсів
cap.release()
cv2.destroyAllWindows()
```

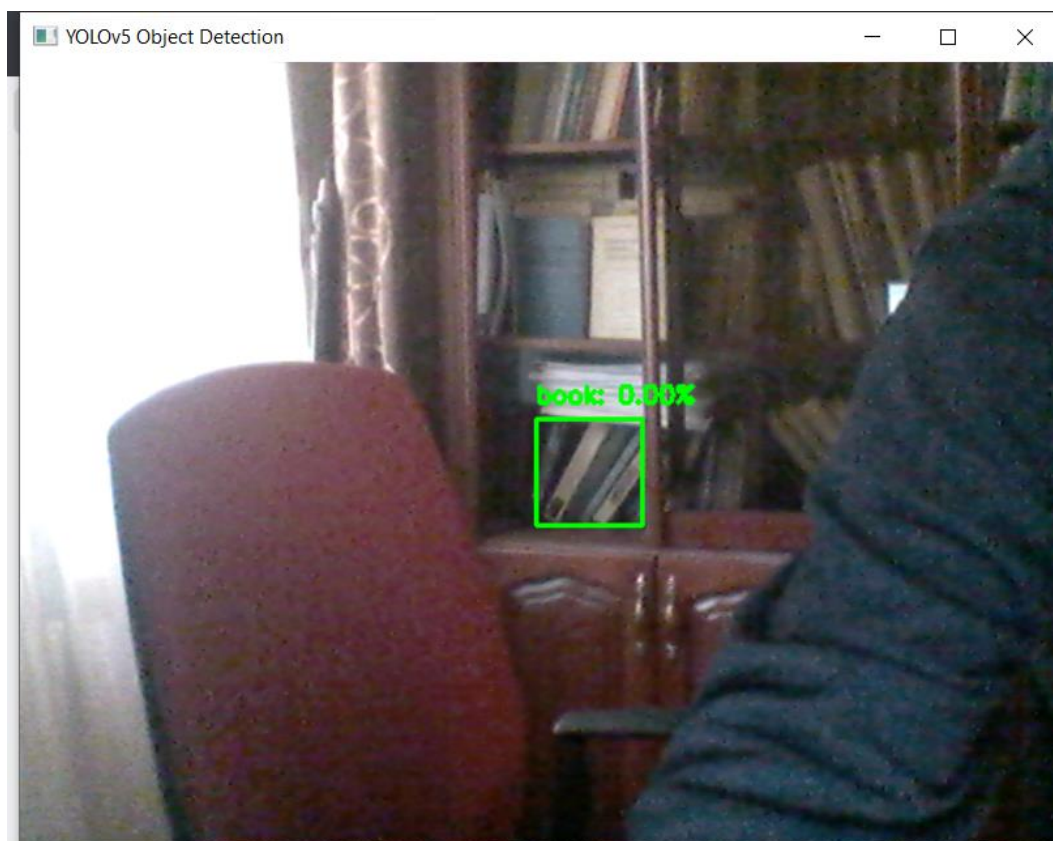
Опис програми

- Завантаження моделі YOLOv5:** Програма завантажує натреновану модель yolo5s, яка призначена для обробки зображень у реальному часі. Ця модель визначає об'єкти на основі попереднього навчання на датасеті COCO, де є 80 класів об'єктів.
- Обробка відео з камери:** Кожен кадр з камери передається в модель для класифікації та розпізнавання об'єктів.
- Розпізнавання об'єктів:** Для кожного об'єкта модель повертає координати рамки, рівень достовірності та назву класу. Ми фільтруємо об'єкти та малюємо прямокутник навколо них із позначенням класу та рівнем достовірності.

4. **Відображення результату:** Кадр із позначками об'єктів відображається в режимі реального часу.
5. **Вихід:** Натискання q закриває програму.

Примітки

- Модель **yolov5s** є найбільш швидкою та меншою за розміром серед моделей **YOLOv5**. Інші моделі, такі як **yolov5m** та **yolov5l**, мають більшу точність, але працюють повільніше.
- Для покращення продуктивності на CPU або GPU можна використати оптимізовані моделі YOLO, як-от **yolov5x**.



Ще один підхід до мультимодального розпізнавання можна реалізувати, об'єднавши виявлення обличчя на відеопотоці з розпізнаванням голосу. Програма

може визначати, коли з'являється нове обличчя в кадрі, а потім записувати та розпізнавати голос користувача, щоб здійснити додаткову перевірку.

- **Приклад програми для мультимодального розпізнавання на відеопотоці та голосовій команді**
- Цей приклад використовує **OpenCV** для обробки відеопотоку та **speech_recognition** для розпізнавання голосу.

[MS LEC8-3]

#Розпізнавання як зображення, так і голосу

```
import cv2
import speech_recognition as sr
import numpy as np
import threading

# Завантаження попередньо навченої моделі для
розпізнавання облич
face_cascade =
cv2.CascadeClassifier(cv2.data.harcascades +
'haarcascade_frontalface_default.xml')

# Функція для розпізнавання голосу
def recognize_speech():
    recognizer = sr.Recognizer()
    with sr.Microphone() as source:
        print("Скажіть щось...")

recognizer.adjust_for_ambient_noise(source)
```

```

        while True:
            try:
                audio = recognizer.listen(source,
timeout=5)
                text =
recognizer.recognize_google(audio, language='uk-
UA')
                print("Ви сказали: " + text)
            except sr.UnknownValueError:
                print("Не вдалося розпізнати
мову.")
            except sr.RequestError:
                print("Помилка сервісу
розпізнавання.")
            except sr.WaitTimeoutError:
                print("Очікування голосу...")

# Функція для розпізнавання обличчя
def recognize_face():
    cap = cv2.VideoCapture(0)

    while True:
        ret, frame = cap.read()
        if not ret:
            break
        gray = cv2.cvtColor(frame,
cv2.COLOR_BGR2GRAY)
        faces =
face_cascade.detectMultiScale(gray,
scaleFactor=1.1, minNeighbors=5, minSize=(30, 30))

        for (x, y, w, h) in faces:
            cv2.rectangle(frame, (x, y), (x + w, y
+ h), (0, 255, 0), 2)

        cv2.imshow('Розпізнавання обличч', frame)

        if cv2.waitKey(1) & 0xFF == ord('q'):
            break

```

```

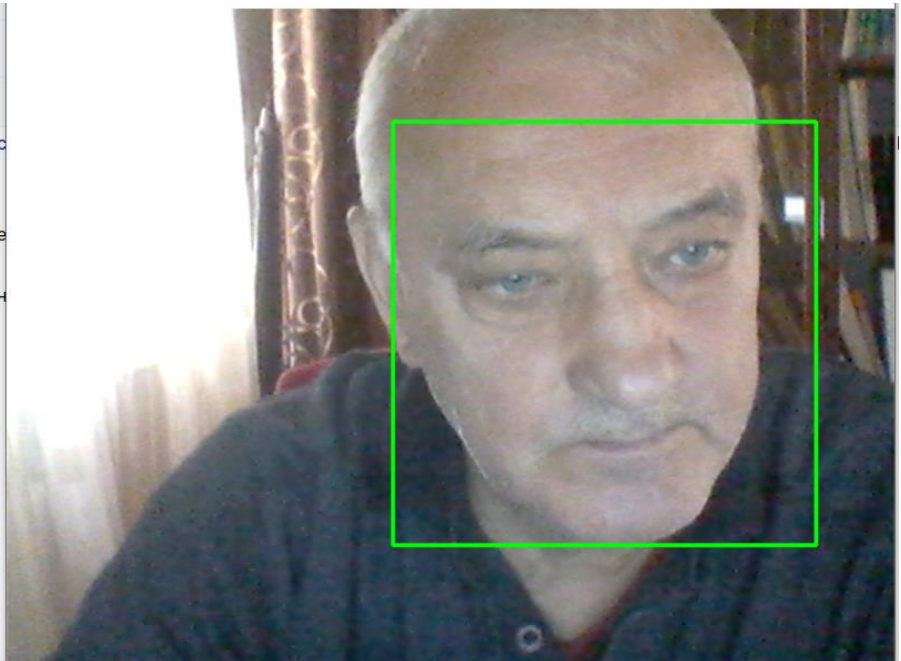
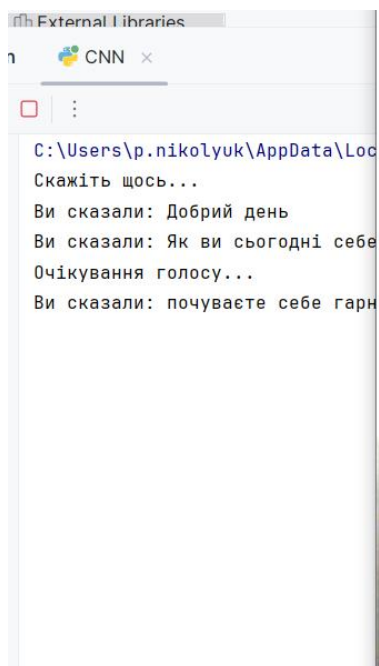
cap.release()
cv2.destroyAllWindows()

# Запуск розпізнавання в двох потоках
if __name__ == "__main__":
    face_thread =
threading.Thread(target=recognize_face)
    speech_thread =
threading.Thread(target=recognize_speech)

    face_thread.start()
    speech_thread.start()

    face_thread.join()
    speech_thread.join()

```



Згорткові нейронні мережі – їх принцип роботи

Згорткові нейронні мережі (Convolutional Neural Networks, CNN) – це тип нейронних мереж, які особливо ефективні для обробки та аналізу зображень, відео, текстів і навіть звукових сигналів. Вони використовують спеціальні згорткові шари, що допомагають автоматично знаходити характерні ознаки у вхідних даних, такі як краєчки, текстури, форми тощо.

Принцип роботи згорткових нейронних мереж

Основні компоненти CNN включають згорткові шари, шари субдискретизації (пулінгу) та повнозв'язні шари. Коротко розглянемо їх роль і принцип роботи.

1. Згортковий шар (Convolutional Layer):

- Основна частина CNN, яка виконує операцію згортки (convolution).
- **Фільтри (ядра):** У кожному згортковому шарі застосовується кілька невеликих фільтрів (наприклад, 3x3 або 5x5), які ковзають по зображенню та виявляють локальні ознаки, як-от контури та текстури. Кожен фільтр шукає певні ознаки та виводить карту ознак (feature map).
- **Згортка:** Кожен фільтр виконує математичну операцію (згортку) з окремою частиною зображення, множачи значення пікселів на ваги фільтра та підсумовуючи результати. Отримані значення формують нову матрицю, яка показує, наскільки чітко певна ознака проявляється в різних частинах зображення.

2. Шар активації (Activation Layer):

- Зазвичай після кожного згорткового шару застосовується функція активації, наприклад **ReLU (Rectified Linear Unit)**, яка відкидає негативні значення, перетворюючи їх на нуль.
- Це допомагає мережі ефективно знаходити нелінійні зв'язки та стабілізує процес навчання.

3. Шар пулінгу (Pooling Layer):

- **Мета:** Зменшити розмірність даних і знизити обчислювальні витрати, зберігаючи при цьому основну інформацію.
- **Максимальний пулінг (Max Pooling):** Найчастіше використовують операцію максимального пулінгу, яка виділяє максимальне значення з кожної ділянки. Це зберігає найяскравіші ознаки та усуває менш значущі дані.
- **Результат:** Після шару пулінгу розмір карти ознак зменшується, що робить мережу менш сприйнятливою до невеликих зсувів на зображенні та підвищує стійкість моделі.

4. Повнозв'язний шар (Fully Connected Layer):

- На завершення згорткових шарів CNN зазвичай використовує повнозв'язні шари, які об'єднують отримані карти ознак і виводять остаточне рішення.
- Цей шар допомагає мережі навчитися комбінувати високорівневі ознаки для створення остаточних класифікацій.

Етапи обробки зображень у CNN

1. **Розпізнавання базових ознак:** Нижні шари CNN знаходять прості ознаки (краї, кути).
2. **Поєднання ознак:** Середні шари починають розпізнавати складніші структури (форми, текстури).
3. **Високорівневі ознаки:** Верхні шари фокусуються на розпізнаванні складних об'єктів (частини обличчя, лапи тварин тощо).
4. **Класифікація:** Останній етап – це визначення, що саме зображено, на основі знайдених ознак. Повнозв'язний шар використовується для остаточного виводу класифікації або іншого результату.

Завдяки такій архітектурі CNN здатні автоматично виділяти найважливіші особливості зображень та використовувати їх для класифікації, сегментації або інших завдань.

Програма Python, що демонструє роботу згорткової нейронної мережі

Щоб продемонструвати роботу згорткової нейронної мережі (CNN) на Python, ми можемо створити просту модель CNN для класифікації зображень із набору даних, наприклад, **MNIST** (набір зображень рукописних цифр) або **CIFAR-10** (набір зображень із 10 класів об'єктів). Ми скористаємося бібліотекою **Keras** (частина TensorFlow), яка забезпечує зручний інтерфейс для створення і навчання нейронних мереж.

Приклад програми на Python для класифікації цифр за допомогою CNN (набору даних MNIST)

[KMM LW 1-1]

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix
from tensorflow import keras
from tensorflow.keras import layers

# 1. Завантаження датасету MNIST
(x_train, y_train), (x_test, y_test) =
keras.datasets.mnist.load_data()

# Масштабування у [0,1]
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

# Перетворення у формат (кількість, висота, ширина,
канали)
x_train = np.expand_dims(x_train, -1)      # (60000, 28, 28,
1)
x_test = np.expand_dims(x_test, -1)        # (10000, 28, 28,
1)

# One-hot кодування міток
y_train_cat = keras.utils.to_categorical(y_train, 10)
y_test_cat = keras.utils.to_categorical(y_test, 10)

# 2. Побудова CNN-моделі
model = keras.Sequential([
```

```

        layers.Conv2D(32, kernel_size=(3, 3),
activation="relu", input_shape=(28, 28, 1)),
        layers.MaxPooling2D(pool_size=(2, 2)),
        layers.Conv2D(64, kernel_size=(3, 3),
activation="relu"),
        layers.MaxPooling2D(pool_size=(2, 2)),
        layers.Flatten(),
        layers.Dense(128, activation="relu"),
        layers.Dense(10, activation="softmax")
])

# -----
# 3. Компіляція моделі
# -----
model.compile(
    optimizer="adam",
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

# -----
# 4. Навчання моделі
# -----
history = model.fit(
    x_train, y_train_cat,
    epochs=5,
    batch_size=128,
    validation_split=0.2,
    verbose=2
)

# -----
# 5. Оцінка на тестових даних
# -----
test_loss, test_acc = model.evaluate(x_test, y_test_cat,
verbose=0)
print(f"\nТочність на тестових даних: {test_acc:.4f}")

# -----
# 6. Графіки точності та втрат
# -----
plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.plot(history.history["accuracy"], label="Навчання")
plt.plot(history.history["val_accuracy"],
label="Валідація")

```

```

plt.title("Точність CNN-моделі")
plt.xlabel("Епоха")
plt.ylabel("Точність")
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(history.history["loss"], label="Навчання")
plt.plot(history.history["val_loss"], label="Валідація")
plt.title("Функція втрат CNN-моделі")
plt.xlabel("Епоха")
plt.ylabel("Втрати")
plt.legend()

plt.show()

# -----
# 7. Матриця плутанини
# -----
y_pred = model.predict(x_test)
y_pred_classes = np.argmax(y_pred, axis=1)

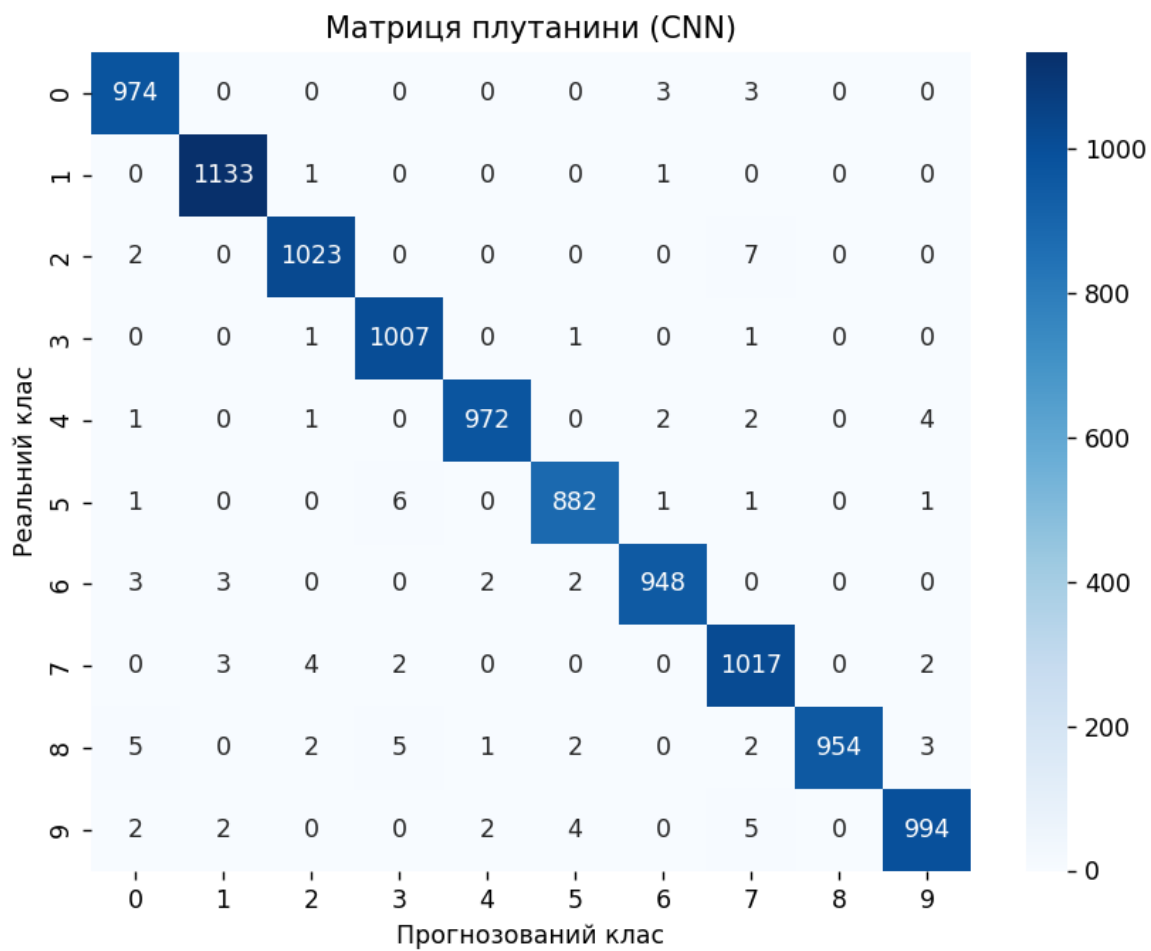
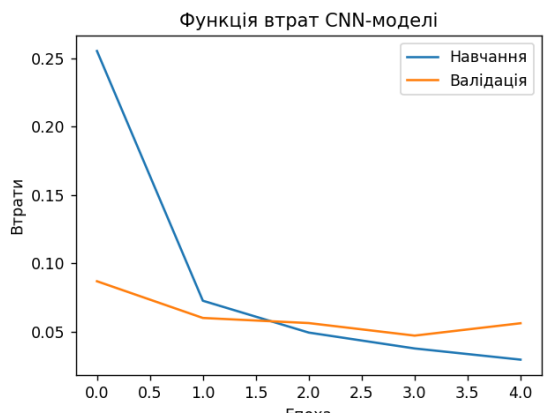
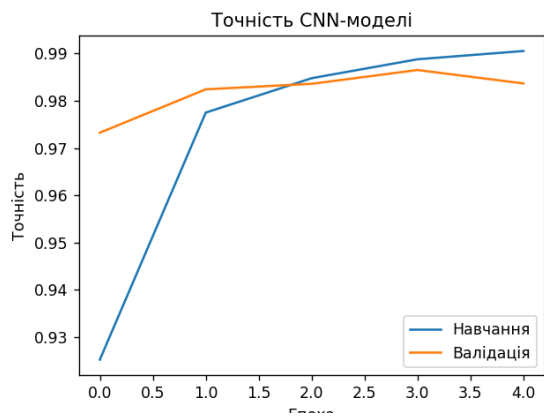
cm = confusion_matrix(y_test, y_pred_classes)

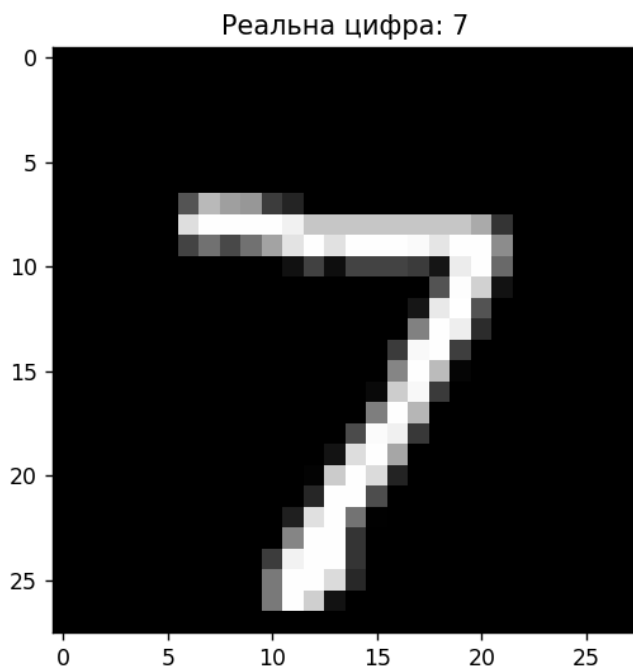
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
plt.title("Матриця плутанини (CNN)")
plt.xlabel("Прогнозований клас")
plt.ylabel("Реальний клас")
plt.show()

# -----
# 8. Приклад розпізнавання
# -----
index = 0
plt.imshow(x_test[index].reshape(28, 28), cmap="gray")
plt.title(f"Реальна цифра: {y_test[index]}")
plt.show()

pred = model.predict(x_test[index:index+1])
print("Ймовірності класів:", pred)
print("Розпізнана цифра:", np.argmax(pred))

```





Використання технологій мультимодального розпізнавання

Мультимодальне розпізнавання – це технологія, яка поєднує інформацію з кількох різних модальностей (зорові, звукові, текстові, сенсорні сигнали тощо) для підвищення точності та надійності розпізнавання. Наприклад, мультимодальні системи можуть використовувати зображення, аудіо, текст або інші джерела інформації для глибшого розуміння та аналізу даних. Це особливо корисно для створення штучного інтелекту (ШІ), здатного працювати в умовах реального світу, де дані можуть бути неоднорідними і неповними.

Використання мультимодального розпізнавання

1. Обробка природної мови та зображень:

- Поєднання текстових і зорових даних дозволяє розпізнавати об'єкти на зображеннях і давати їм текстові описи (наприклад, технології автоматичного генерування підписів до зображень).

- Наприклад, система бачить зображення з об'єктом і розпізнає його за допомогою зорових даних, одночасно ідентифікуючи текстову інформацію (назви, підписи, контекст).

2. Аудіовізуальне розпізнавання:

- Використовується для розпізнавання емоцій або визначення особи. При цьому система аналізує як візуальну інформацію (міміку, жести), так і аудіодані (тональність, ритм).
- У голосових помічниках (наприклад, Siri, Google Assistant) аудіо використовується для обробки команд, а візуальні дані (якщо є камера) можуть додавати інформацію про місцезнаходження користувача чи його емоційний стан.

3. Розпізнавання емоцій у взаємодії з користувачем:

- Мультимодальні системи можуть оцінювати емоційний стан користувача за виразом обличчя, голосом, жестами або навіть пульсом. Наприклад, у медицині такі системи можуть допомогти лікарям відстежувати стан пацієнта.
- У маркетингу мультимодальні системи можуть аналізувати реакцію користувачів на продукти через аудіо- та відеозаписи, щоб зрозуміти їхню зацікавленість.

4. Аналіз ситуації та дій у відеоспостереженні:

- В автоматизованих системах безпеки використовують мультимодальні технології для розпізнавання загроз і ненормальної поведінки, поєднуючи відеодані з сенсорними показниками (наприклад, температури, звуку, руху).
- У транспорті ці системи допомагають виявляти перешкоди, аналізувати поведінку водія (зоровий і звуковий моніторинг) і прогнозувати ризики аварій.

5. Віртуальна та доповнена реальність (VR/AR):

- У додатках VR та AR мультимодальні технології дозволяють користувачам взаємодіяти з віртуальним середовищем за допомогою

жестів, голосу та зорового контакту. Наприклад, у віртуальних тренажерах для навчання це створює ефект реалістичної взаємодії.

6. Медичні додатки та діагностика:

- Мультимодальне розпізнавання даних використовується в медичних системах, які аналізують дані МРТ, КТ, електрокардіограм і текстових записів лікарів одночасно для отримання комплексної картини стану пацієнта. Це дозволяє покращити точність діагнозу.

Переваги мультимодального розпізнавання

- **Підвищення точності:** Поєднання різних джерел інформації дозволяє компенсувати недоліки кожної модальності.
- **Надійність:** Мультимодальні системи більш стійкі до шуму та неповних даних, оскільки вони можуть опиратися на інші джерела інформації.
- **Гнучкість:** Більша кількість можливих сценаріїв використання в реальному житті, де дані можуть бути непередбачуваними і мати кілька форм.

Мультимодальні технології вже застосовуються у багатьох сферах, і їх використання швидко зростає, адже вони значно покращують здатність систем ШІ розуміти складні взаємодії та контексти.

[MS LEC8-5]

#У порівнянні з попередньою програмою додано обробку зображень для покращення розпізнавання вночі. Використовується перетворення в градації сірого та підсилення контрасту (`cv2.equalizeHist`). Це допоможе покращити точність розпізнавання об'єктів дронами в умовах низького освітлення. Можна також додати тепловізійний аналіз або інші методи!

```

import torch
import torchvision.transforms as transforms
from PIL import Image
from transformers import BlipProcessor,
BlipForConditionalGeneration, DetrImageProcessor,
DetrForObjectDetection
import cv2
import numpy as np

# Завантаження моделей
blip_processor =
BlipProcessor.from_pretrained("Salesforce/blip-
image-captioning-base")
blip_model =
BlipForConditionalGeneration.from_pretrained("Sale
sforce/blip-image-captioning-base")

detr_processor =
DetrImageProcessor.from_pretrained("facebook/detr-
resnet-50")
detr_model =
DetrForObjectDetection.from_pretrained("facebook/d
etr-resnet-50")

def preprocess_night_image(image_path):
    image = cv2.imread(image_path)
    if image is None:
        raise FileNotFoundError(f"Файл
{image_path} не знайдено.")

    # Перетворення в градації сірого та підсилення
контрасту
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    enhanced = cv2.equalizeHist(gray)

    # Перетворення назад у формат RGB
    image_rgb = cv2.cvtColor(enhanced,
cv2.COLOR_GRAY2RGB)
    return Image.fromarray(image_rgb)

```

```

def generate_caption(image_path):
    image = preprocess_night_image(image_path)
    inputs = blip_processor(image,
return_tensors="pt")
    with torch.no_grad():
        caption = blip_model.generate(**inputs)
    return blip_processor.batch_decode(caption,
skip_special_tokens=True) [0]

def detect_objects(image_path):
    image = preprocess_night_image(image_path)
    inputs = detr_processor(images=image,
return_tensors="pt")
    with torch.no_grad():
        outputs = detr_model(**inputs)

    # Отримання результатів розпізнавання без
    # урахування фону
    results = []
    for logits, boxes in zip(outputs.logits[0],
outputs.pred_boxes[0]):
        score =
torch.nn.functional.softmax(logits, dim=-1).max()
        label_id = logits.argmax().item()

        # Пропустити фон (ID 91)
        if label_id == 91:
            continue

        label =
detr_model.config.id2label[label_id]
        if score > 0.5: # Фільтр за порогом
# ПЕВНЕНОСТІ
            results.append((label, score.item(),
boxes.tolist()))
    return results

if __name__ == "__main__":

```

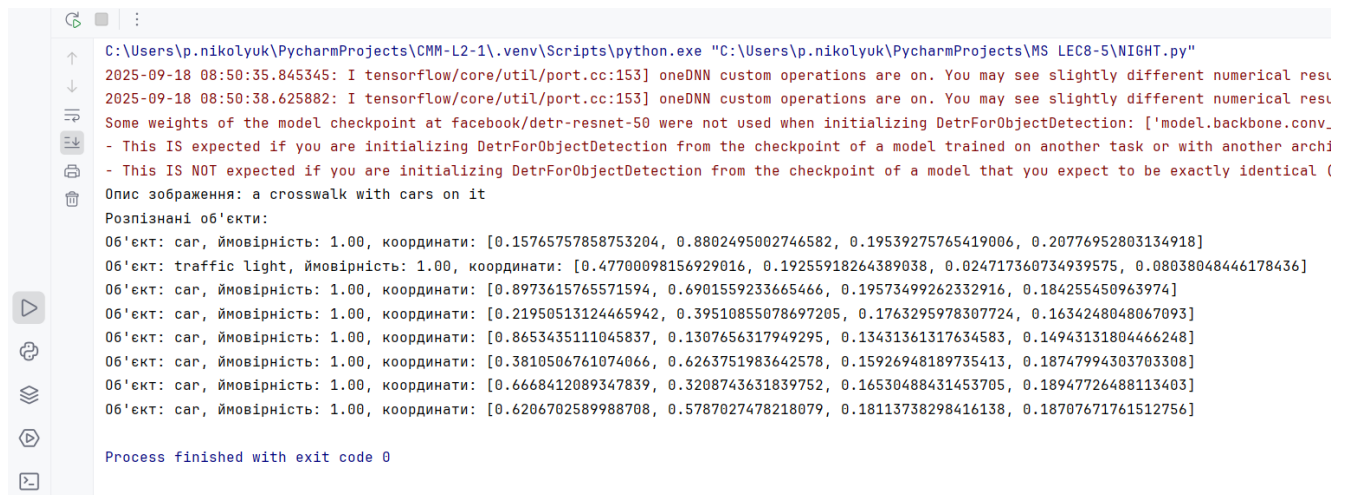
```

image_path = "image.jpg" # Вкажіть шлях до
зображення

caption = generate_caption(image_path)
objects = detect_objects(image_path)

print("Опис зображення:", caption)
print("Розпізнані об'єкти:")
for obj in objects:
    print(f"Об'єкт: {obj[0]}, ймовірність:
{obj[1]:.2f}, координати: {obj[2]}")

```



```

C:\Users\p.nikolyuk\PycharmProjects\CMM-L2-1\.venv\Scripts\python.exe "C:\Users\p.nikolyuk\PycharmProjects\MS LEC8-5\NI6HT.py"
2025-09-18 08:50:35.845345: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical resu
2025-09-18 08:50:38.625882: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical resu
Some weights of the model checkpoint at facebook/detr-resnet-50 were not used when initializing DetrForObjectDetection: ['model.backbone.conv.
- This IS expected if you are initializing DetrForObjectDetection from the checkpoint of a model trained on another task or with another archi
- This IS NOT expected if you are initializing DetrForObjectDetection from the checkpoint of a model that you expect to be exactly identical (
Опис зображення: a crosswalk with cars on it
Розпізнані об'єкти:
Об'єкт: car, ймовірність: 1.00, координати: [0.15765757858753204, 0.8802495002746582, 0.19539275765419006, 0.20776952803134918]
Об'єкт: traffic light, ймовірність: 1.00, координати: [0.47700098156929016, 0.19255918264389038, 0.024717360734939575, 0.08038048446178436]
Об'єкт: car, ймовірність: 1.00, координати: [0.8973615765571594, 0.6901559233665466, 0.19573499262332916, 0.184255450963974]
Об'єкт: car, ймовірність: 1.00, координати: [0.21950513124465942, 0.39510855078697205, 0.1763295978307724, 0.1634248048067093]
Об'єкт: car, ймовірність: 1.00, координати: [0.865343511045837, 0.1307656317949295, 0.13431361317634583, 0.14943131804466248]
Об'єкт: car, ймовірність: 1.00, координати: [0.3810506761074066, 0.6263751983642578, 0.15926948189735413, 0.18747994303703308]
Об'єкт: car, ймовірність: 1.00, координати: [0.6668412089347839, 0.3208743631839752, 0.16530488431453705, 0.18947726488113403]
Об'єкт: car, ймовірність: 1.00, координати: [0.6206702589988708, 0.5787027478218079, 0.18113738298416138, 0.18707671761512756]

Process finished with exit code 0

```